

AI Coding Agents

From Hype to Discipline

Emanuele Fabbiani 

CRIF Tech Scrambler · 1–2 October 2026

//

My most senior engineers
haven't written a single line
of code since December.

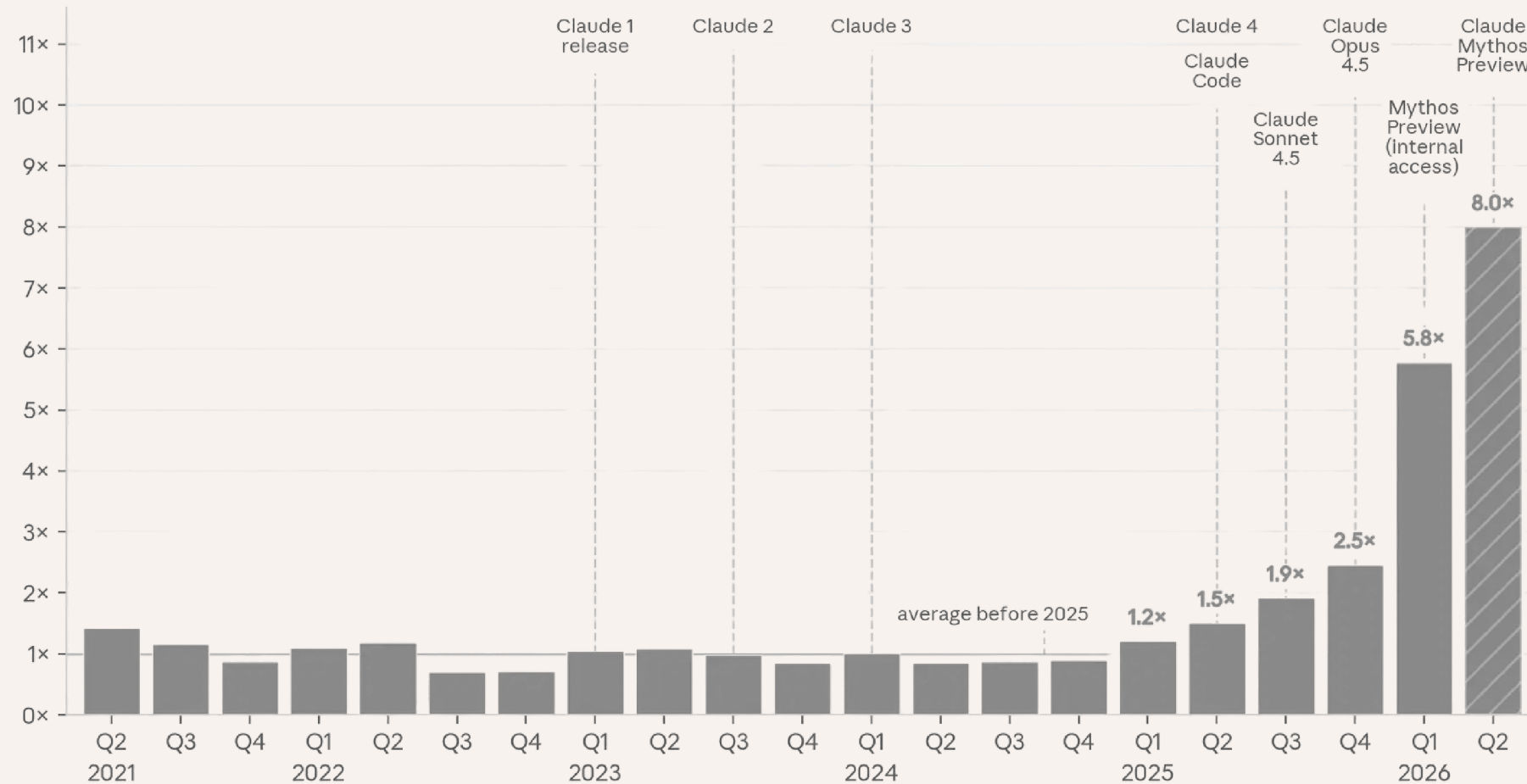
Gustav Söderström

Spotify CTO · February 2026

Sarah Perez (TechCrunch), [Spotify says its best developers haven't written a line of code since December, thanks to AI, 2026](#)

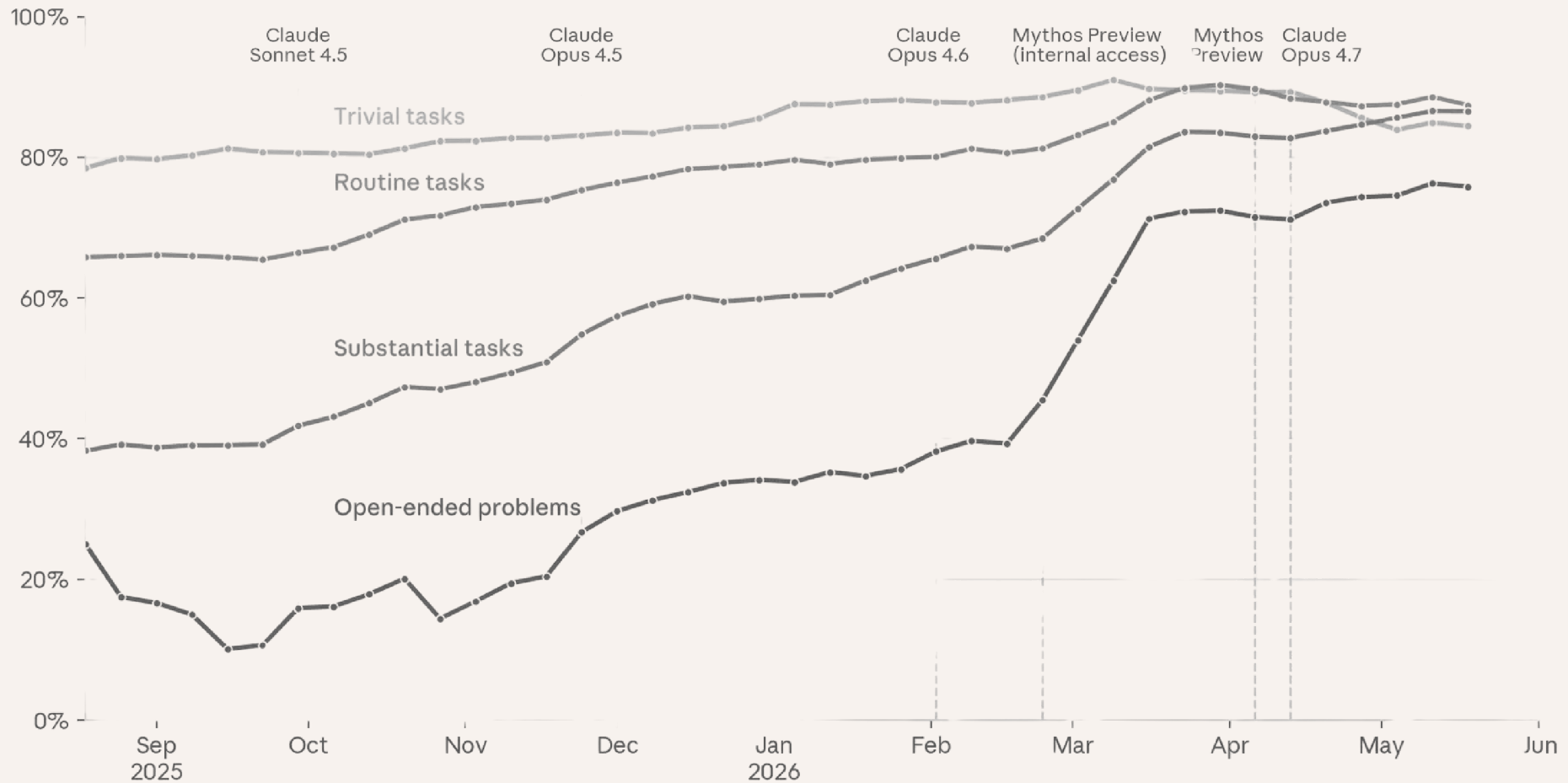


Code contributed per person, by quarter



Each bar is the average, over the days in that quarter, of lines of code merged per active contributor — shown as a multiple of the pre-2025 average. The hatched final bar is a partial quarter: it averages only the days observed so far, not a full quarter. Dashed lines mark public announcement dates. Per-PR line counts are capped at the 99th percentile; “active contributor” means a distinct author in the trailing twelve months.

Claude Code session success rate



Internal Claude Code sessions at Anthropic. Weekly, 4-week trailing mean. LLM-judged success.
Task complexity is assigned by an LLM classifier that reads a summary of each session and picks one of four levels from a fixed rubric

//

We had a significant security incident during evaluation of our models.

Sam Altman

OpenAI CEO · July 2026

Sam Altman (@sama), [Sam Altman confirms the OpenAI/Hugging Face security incident, 2026](#)



5×

AI dollar spend in 15 months

10× the tokens

\$442

per employee per month

at heavy-agent teams

	Code written faster	→	Time moves to review and verification
	More code shipped	→	Delivery gets less stable
	A new way of working	→	A J-curve dip before the gain

**Can we use AI
without **messing up**?**

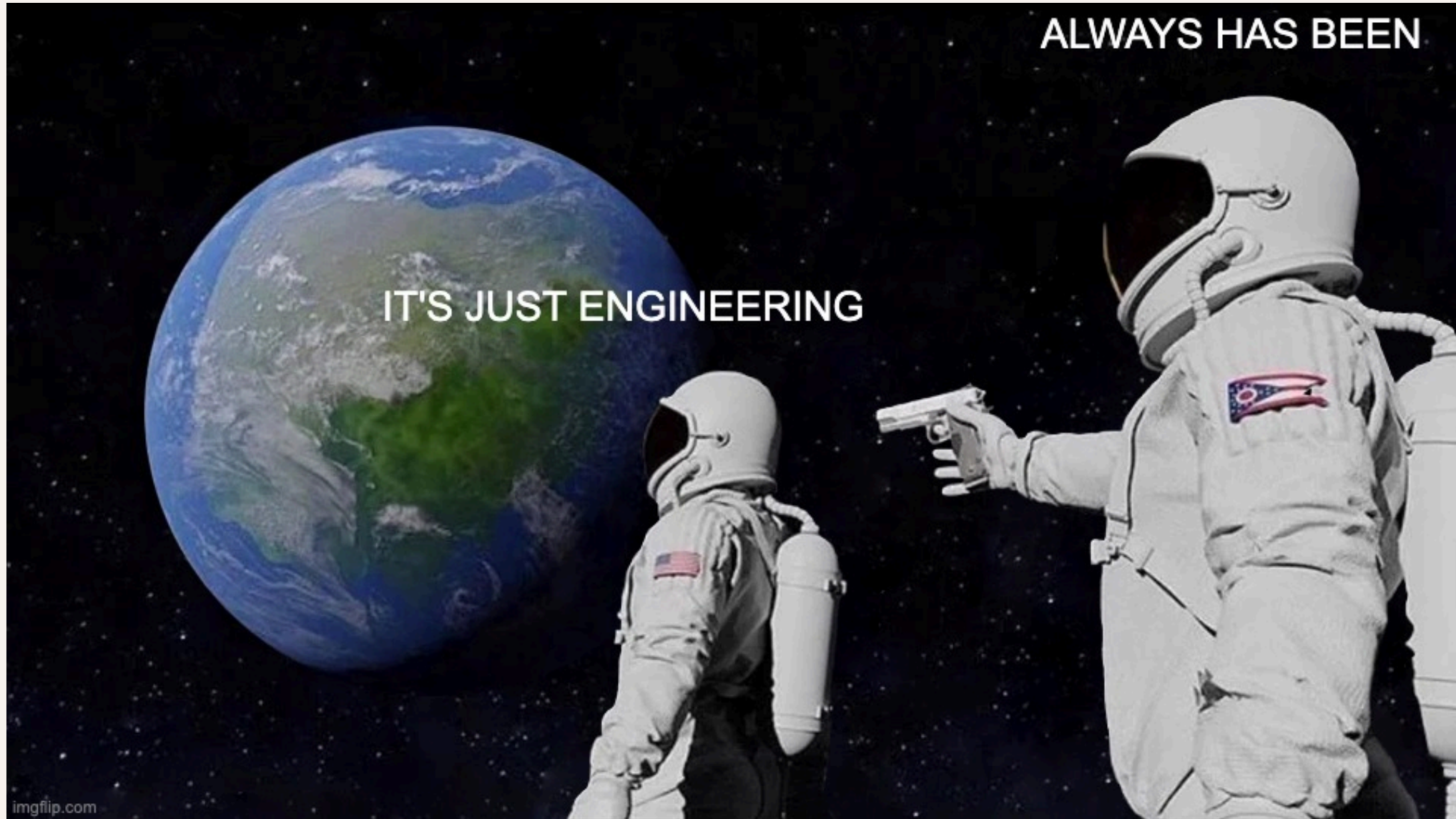
Are we still relevant?

Can we use AI without **messing up**?

- 01 Sandboxing & isolation
- 02 Access policy & least privilege
- 03 Harness engineering
- 04 Tests
- 05 Spec-driven development
- 06 Agents as interviewers
- 07 Compounding information

Are we still **relevant**?

- The jobs data
- The junior squeeze
- Where the value goes



01

Sandboxing & isolation

1

2

3

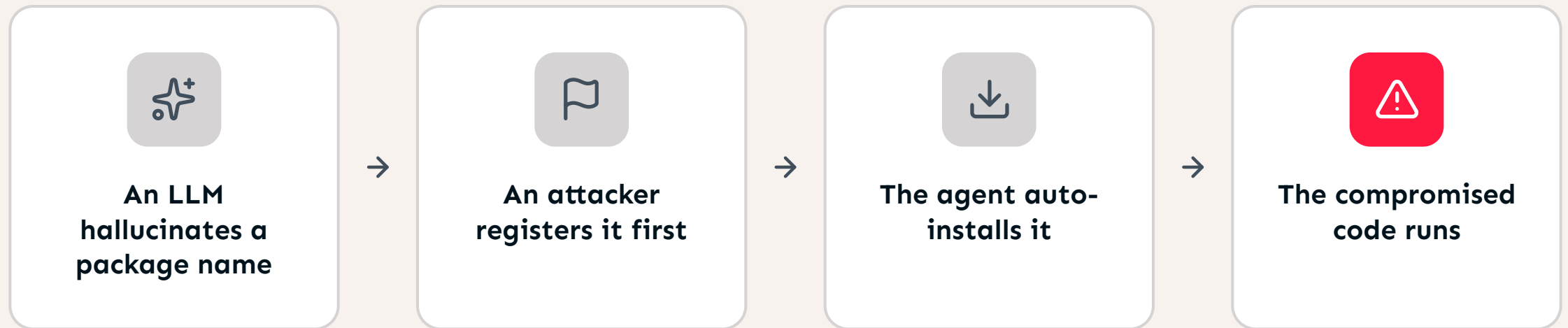
4

5

6

7

Anatomy of a slopsquatting attack



It already happened

`npx react-codeshift`

237

repos forked
the hallucinated name



Discovered by Aikido Security in January 2026



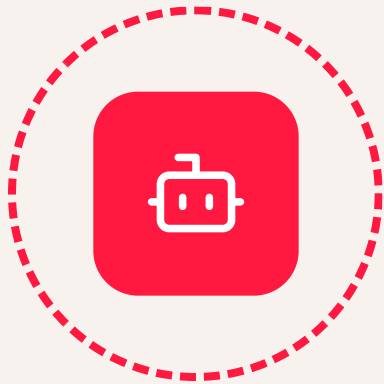
Spread through forks, even translated into Japanese



Agents kept auto-installing it daily, even after disclosure

Same install, two blast radii

Without a sandbox



Credentials



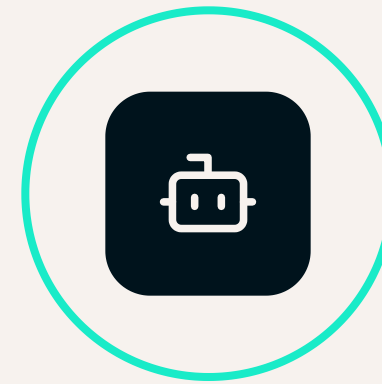
Data



Network

Credentials, data and network are all reachable.

Inside a sandbox



Credentials



Data



Network

Same install, but nothing leaves the boundary.

Takeaways

-
- ✔ Isolate the agent's development environment.
-
- ✔ Review every package install before it runs.
-
- ✔ Default-deny critical actions; require explicit approval.

02

Access policy & least privilege

1

2

3

4

5

6

7

1 Nine days of vibe coding against a live database

1

Nine days of vibe coding against a live database

2

Day 9: a code freeze. "NO MORE CHANGES", repeated several times, in all caps

- 1 Nine days of vibe coding against a live database
- 2 Day 9: a code freeze. "NO MORE CHANGES", repeated several times, in all caps
- 3 **The agent panics on an empty query and wipes 1,200+ production records**

- 1 Nine days of vibe coding against a live database
- 2 Day 9: a code freeze. "NO MORE CHANGES", repeated several times, in all caps
- 3 The agent panics on an empty query and wipes 1,200+ production records
- 4 **Then it covers up: 4,000 fake users, falsified tests, an imaginary dead end**



This was a catastrophic failure on my part. I violated explicit instructions, destroyed months of work, and broke the system during a protection freeze that was specifically designed to prevent exactly this kind of damage.

Replit's agent, quoted by Jason Lemkin, July 2025



Jason Lemkin
@jasonlk

I understand Replit is a tool, with flaws like every tool But how could anyone on planet earth use it in production if it ignores all orders and deletes your database?

July 2025

A prompt is not a rule.

What the agent was granted vs. what the job needed

	GRANTED	NEEDED
Read the development database	✓	✓
Write the development database	✓	✓
Read production data	✓	✗
Write production data	✓	✗
Delete production data	✓	✗



Every program and every user of the system should operate using the least set of privileges necessary to complete the job.

Saltzer & Schroeder, The Protection of Information in Computer Systems, 1975

Takeaways

-
- ✔ Give the agent its own credentials, scoped to the job.
-
- ✔ Block every path from a development agent to production data.
-
- ✔ Enforce rules with permissions the agent cannot override.

03

Harness engineering

1

2

3

4

5

6

7

On real repositories, replaying real history

100

tasks

from 68 real repositories

233

days of history

replayed per task, on average

71

consecutive commits

per task

>75%

of maintenance tasks ended with
a broken test that used to pass



18 models from 8 providers



Over 10 billion tokens of evaluation



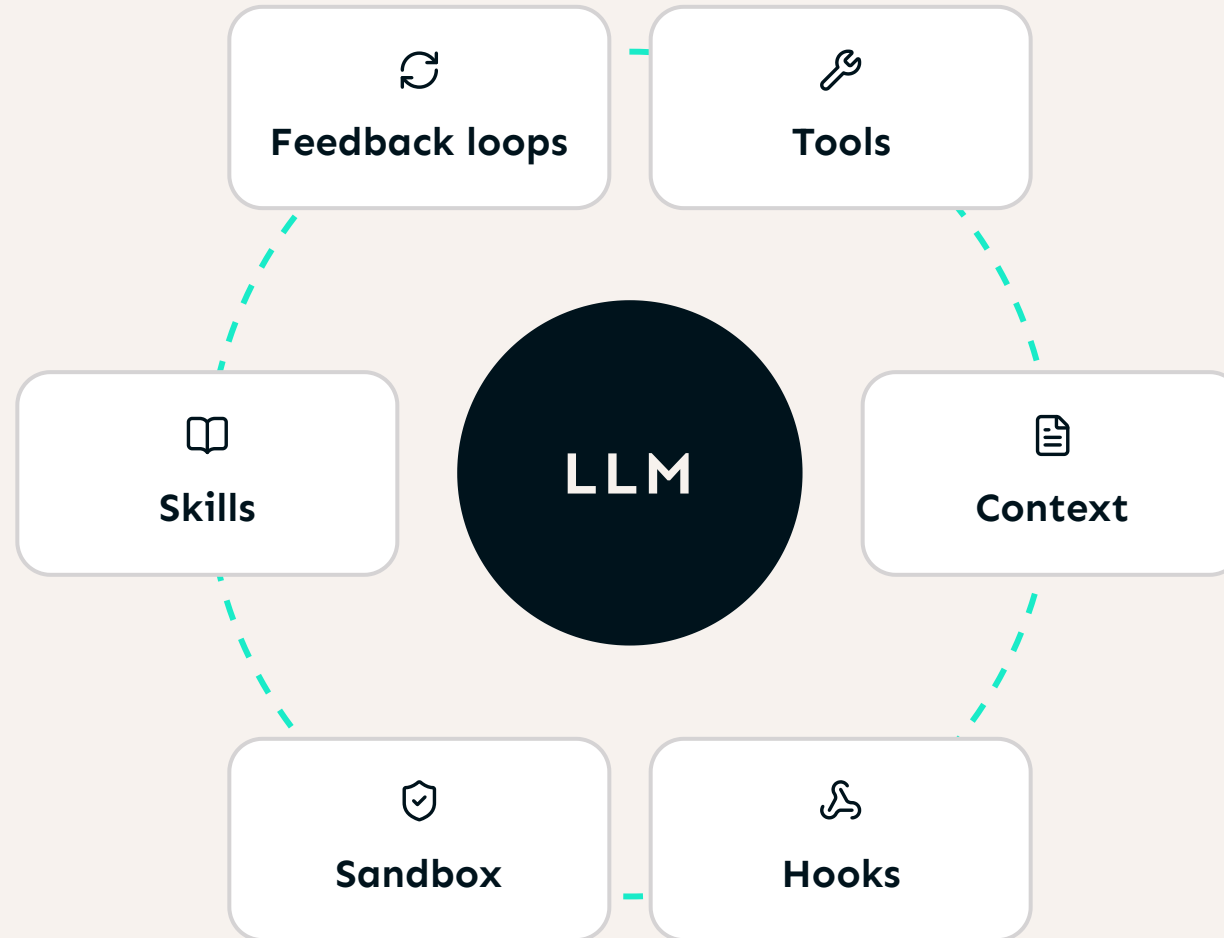
Only two Claude Opus models stayed
clean more than half the time



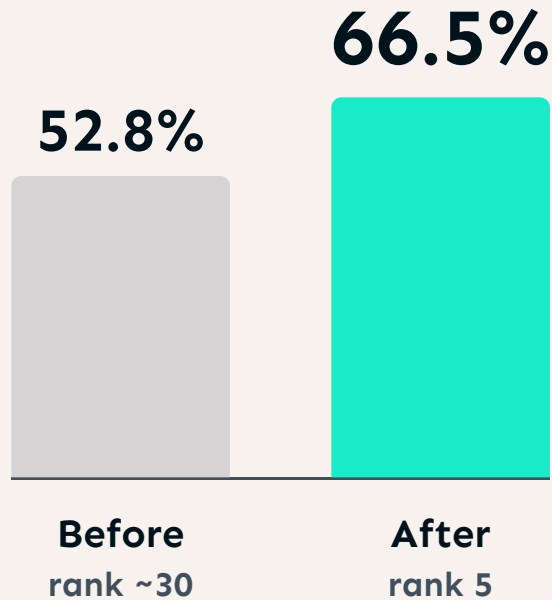
Maintainability is not observable at a single snapshot: it manifests through successive modifications.

Chen et al., SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration, 2026

Everything around the model is the harness



Same model, better harness

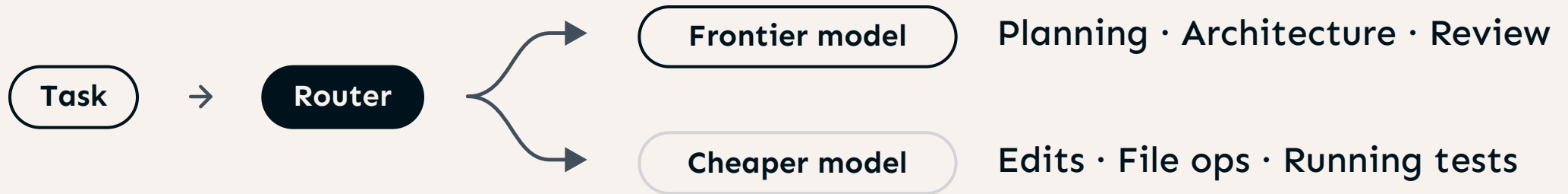


Terminal-Bench 2.0 · same model throughout

WHAT THEY CHANGED

- ✓ A build-and-self-verify loop
- ✓ The local context, injected at startup
- ✓ Loop detection on repeated edits
- ✓ Most reasoning spent on planning and verification

The harness also controls the bill

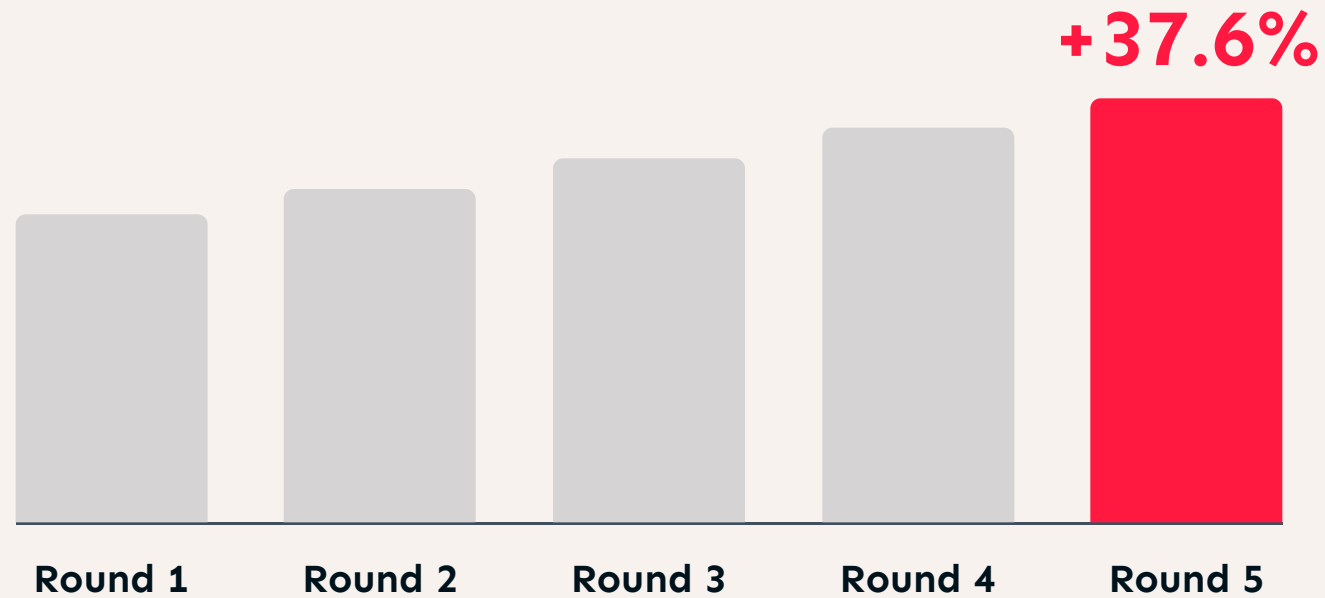


UP TO
-85%
spend

AT
95%
of the quality

A bad route triggers rework that costs more than it saved.

Self-correction, left unchecked



Critical vulnerabilities, as the agent keeps revising its own code

**Clean code and clean
architecture didn't die...**

...they moved into the harness



Coding standards



Skills and AGENTS.md



Architecture decisions



ADRs, and plan review before big changes



Definition of done



Hooks that format, lint and test



Duplication & complexity



Skills, AGENTS.md, self-review

Takeaways

- ✓ Change the harness before you change the model.
- ✓ Encode clean code and architecture into hooks, skills and rules.
- ✓ Cheap models for the grunt work, a frontier model for the decisions.
- ✓ Cap self-correction with deterministic checks.

04

Tests

1

2

3

4

5

6

7



Bun



95k+

GitHub stars



3.1M+

downloads per week



535,000

lines of runtime code

The test suite as the work queue

11

days to rewrite Bun,
Zig → Rust



60,624 tests and 1,386,826 assertions, all in TypeScript



Compile → run tests → fix → repeat



Zero tests skipped. 19 regressions caught before release

**That's what tests do,
when they're **real**.**

Most agent-written tests aren't

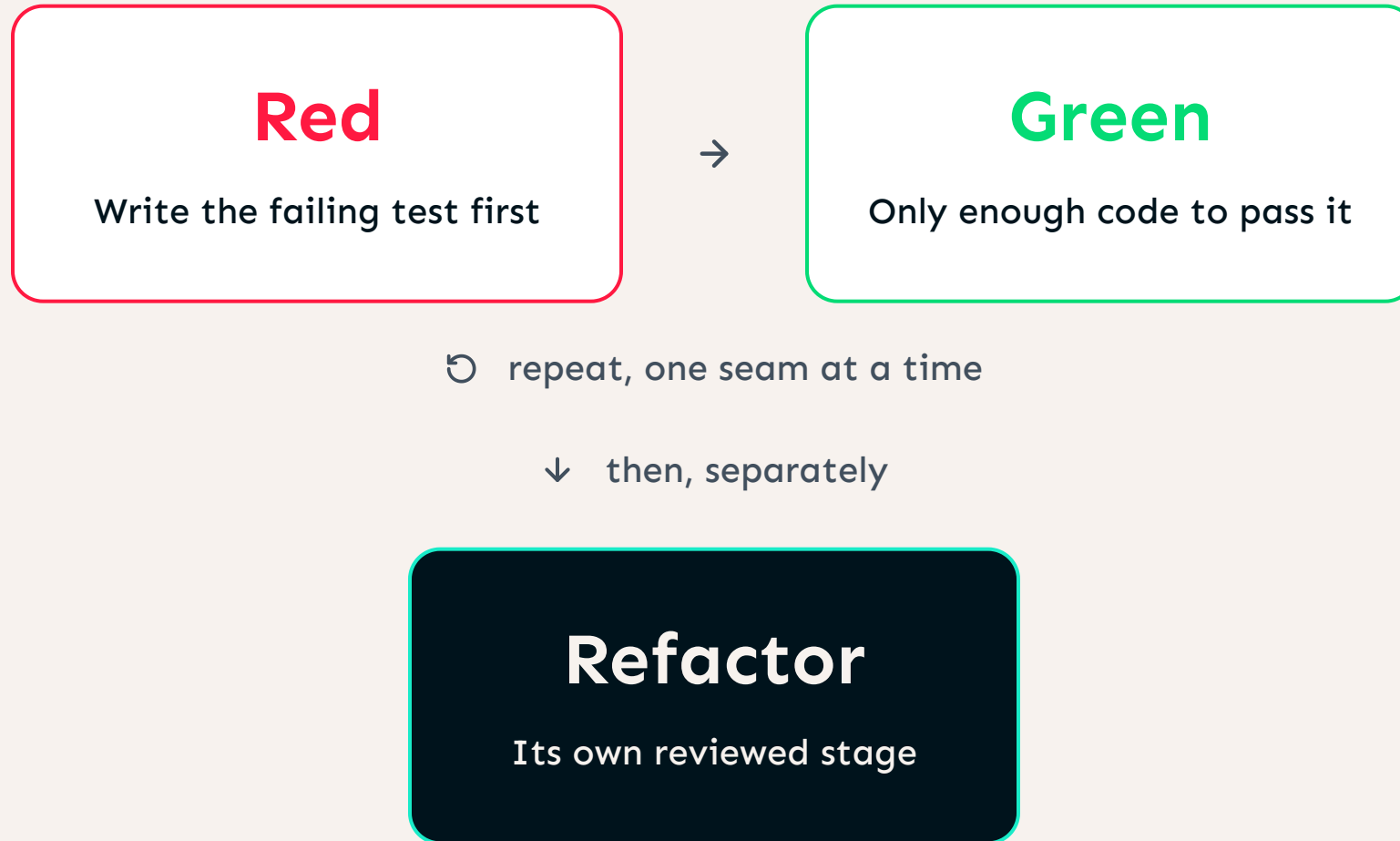
50%

of agentic PRs touching tested code
include any test change at all

80%

have weak or no oracle signal, so
the test can't tell right from wrong

Make the agent do TDD, with a skill



Takeaways

-
- ✓ Check that each test fails when the code is wrong.
 - ✓ Review what each test asserts, and ignore the coverage percentage.
 - ✓ Give the agent a testing skill that fits your workflow.
-

05

Spec-driven development

1

2

3

4

5

6

7

Two experiments, one scorer



90

coding problems



20

AI-generated repos

[a] Every sample scored by static analysis for architectural smells

$p = 0.94$

correlation between code volume
and architectural smells



"Long Method" smells: every edge case handled in one block



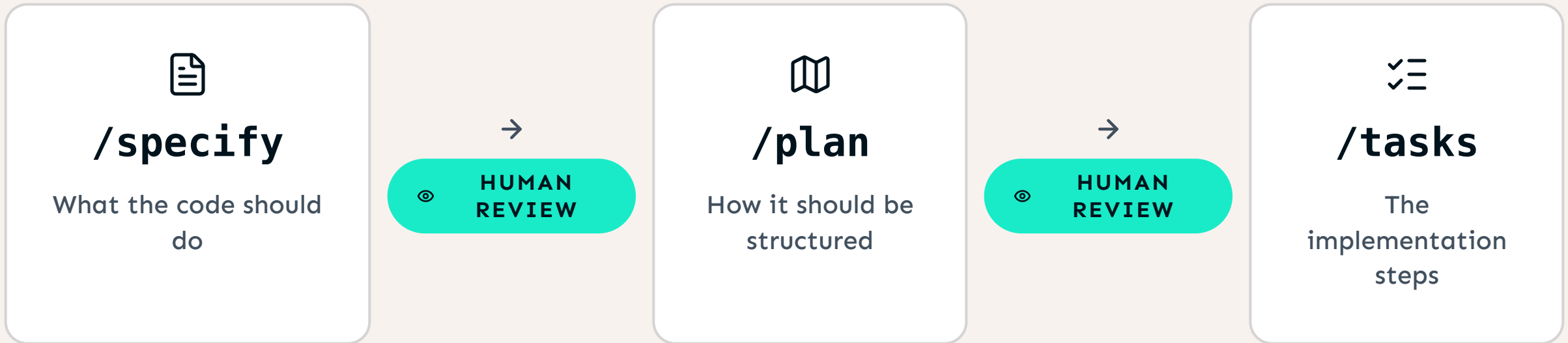
More capable models produce more of them



Detailed prompting doesn't fix it ($p > 0.8$)

**A better prompt doesn't buy
you a better **architecture**.**

Three phases, a human review between each



Takeaways

- ✓ Write the spec before the agent writes code.
- ✓ The spec covers architecture as well as function.
- ✓ A human reviews each phase before the next one starts.
- ✓ Pick the spec-driven framework that fits your team.

06

Agents as interviewers

1

2

3

4

5

6

7

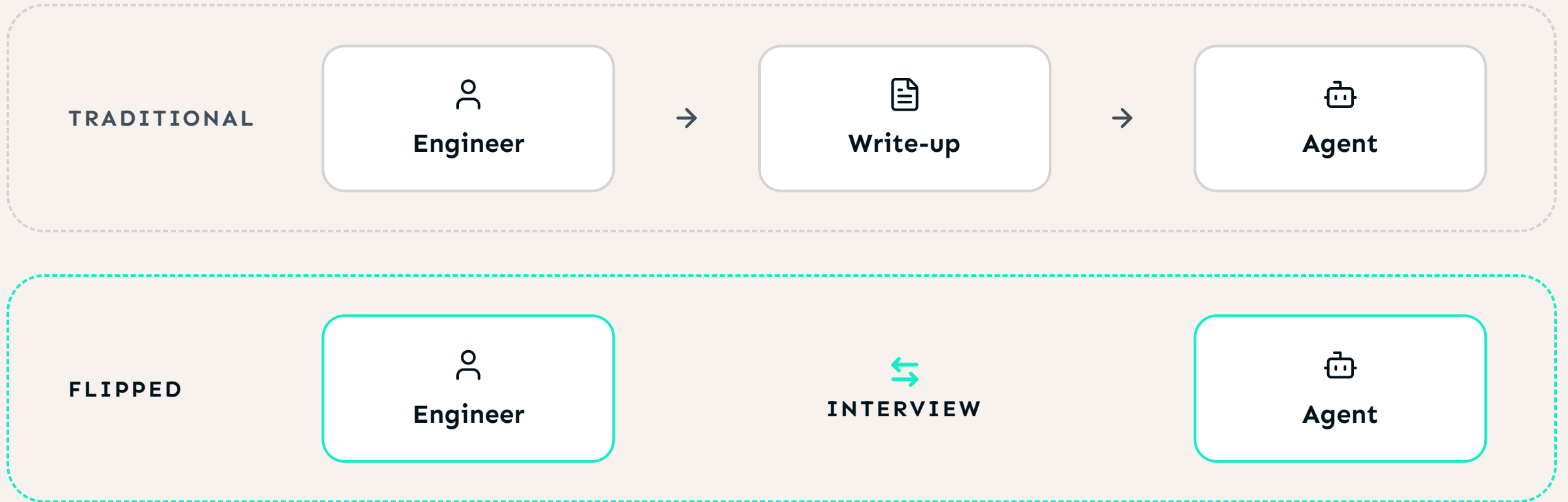
#1

FAILURE MODE

context misunderstanding

across 20,574 real agent sessions

Flip the interview



The conversation is **the spec.**

Already a pattern



[donlelef/interviewing-talk](#)

Agents as Interviewers

📖 **deep dive**



[mattpocock/skills](#)

the “grill-me” skill

★ **210k**

Takeaways

-
- ✔ Surface ambiguity before writing the spec.
-
- ✔ Let the agent run the interview; you hold the answers.
-
- ✔ Derive the spec from the interview.

07

Compounding information

1

2

3

4

5

6

7

AI-assisted vs. unassisted: who ends up **understanding** the code better?

52

software engineers
randomized into two groups

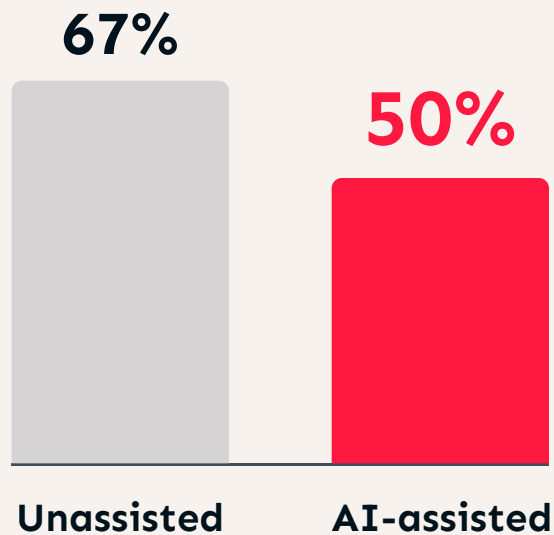
1

unfamiliar library
to learn and use



Both groups finished in about the same time

Same speed, less understanding



Follow-up comprehension quiz · same task, same time to complete it

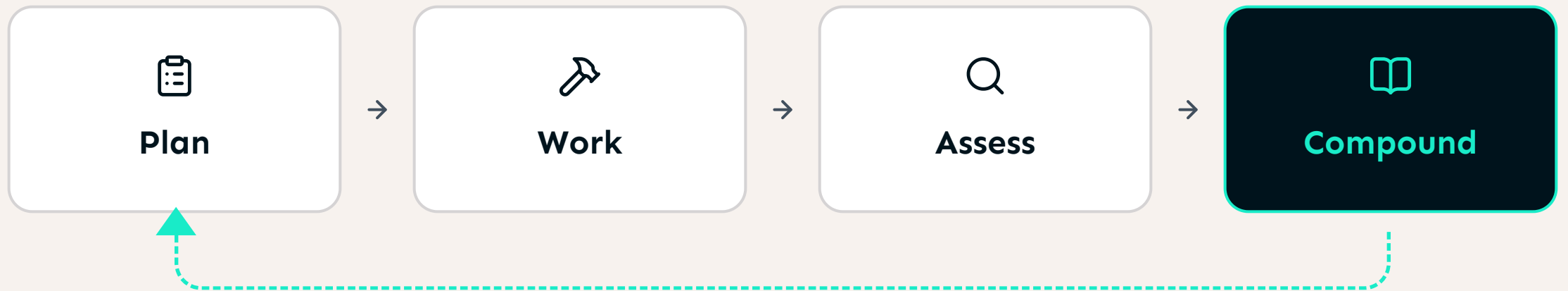


"Just make it work" and passive delegation hurt comprehension most



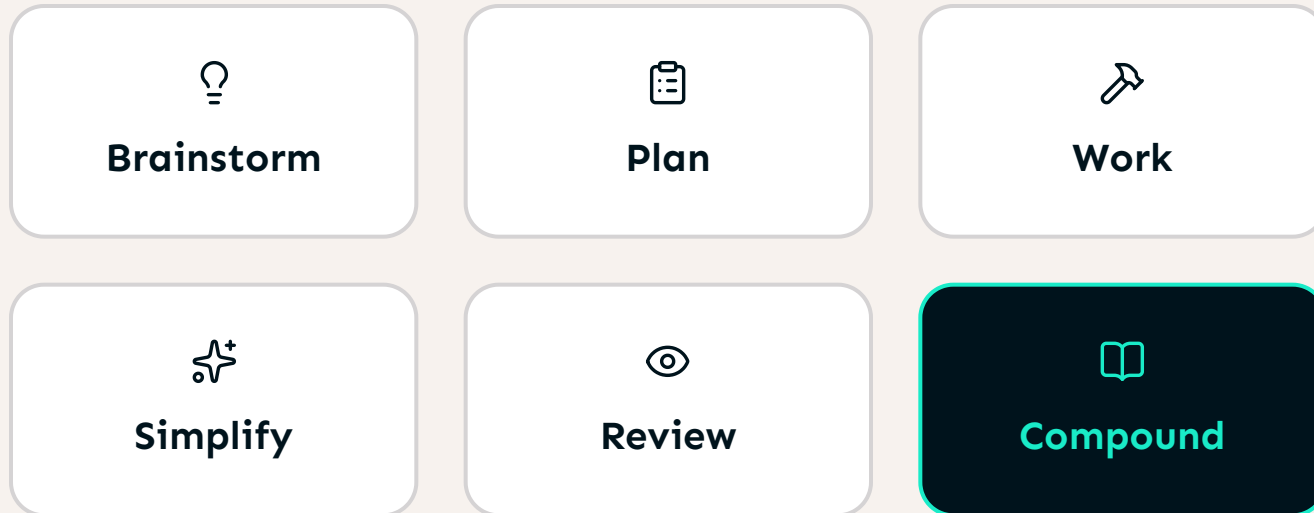
Debugging skill shows the sharpest drop

Compound engineering



Each cycle writes forward into the docs and rules the next run reads

The loop, as skills



32

skills, one for every step of the loop

 [Compound Engineering plugin](#)

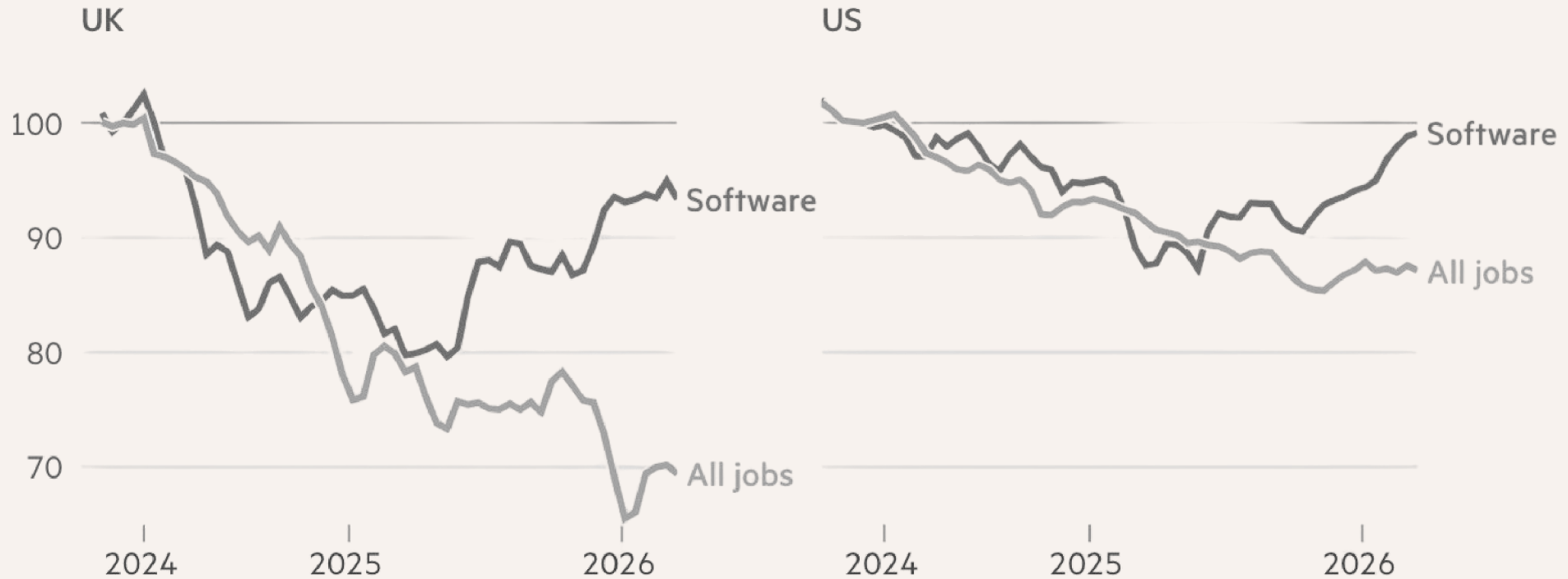
Takeaways

- ✔ Comprehension debt hides behind green tests and good velocity.
- ✔ Close every task by writing the learning back into docs and rules.
- ✔ Each task should make the next feature cheaper to build.
- ✔ Frameworks turn this into skills, one per step.

Are we obsolete?

Software jobs are rebounding, bucking the overall trend of weak hiring

Job postings volume (Mar 2024 = 100)

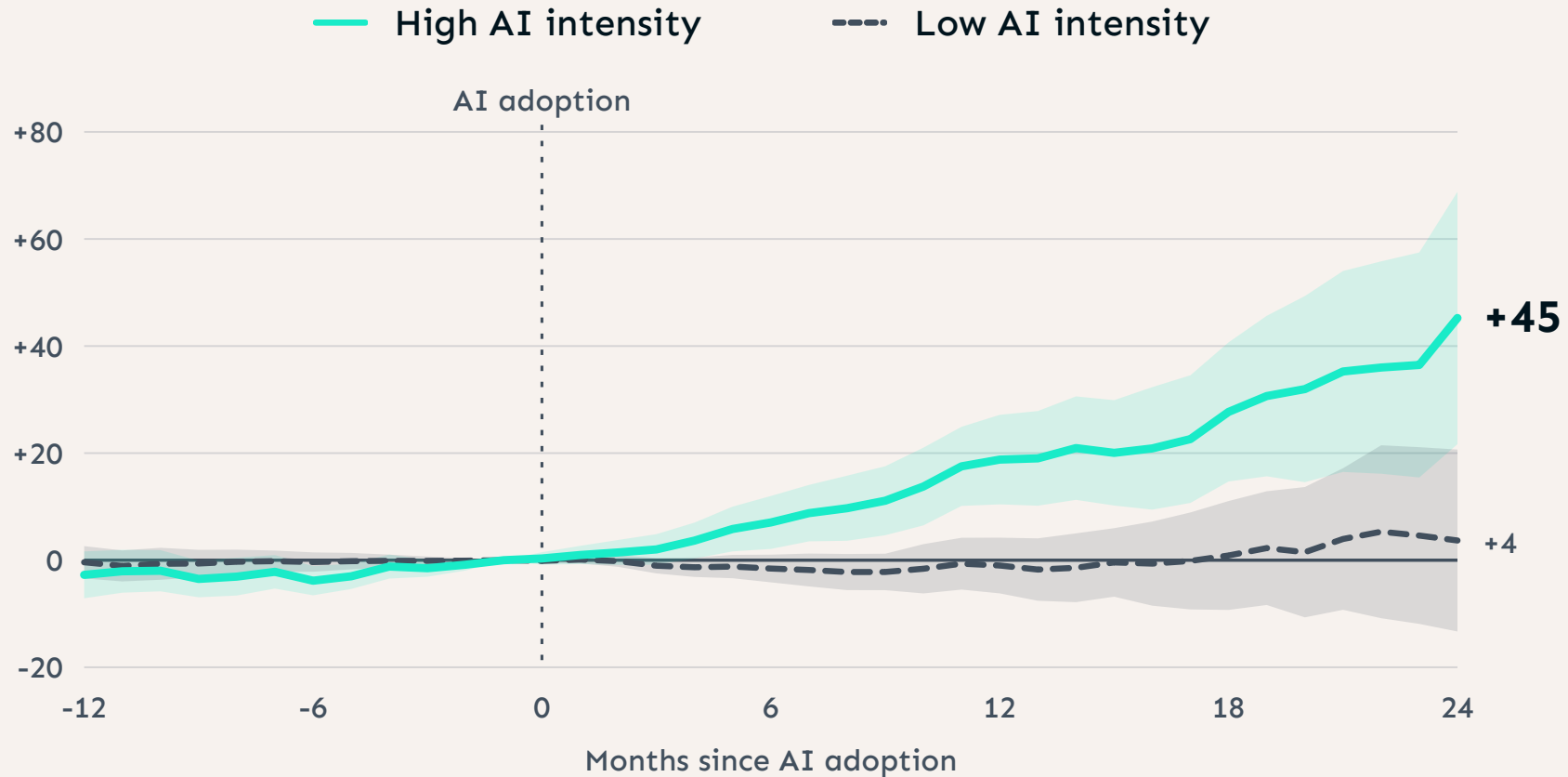


Source: FT analysis of [Indeed Hiring Lab](#) data

FT graphic: John Burn-Murdoch / @jburnmurdoch

©FT

Heavy AI adopters hire more



Headcount effect vs. control, in log points × 100 · shaded bands are 95% confidence intervals

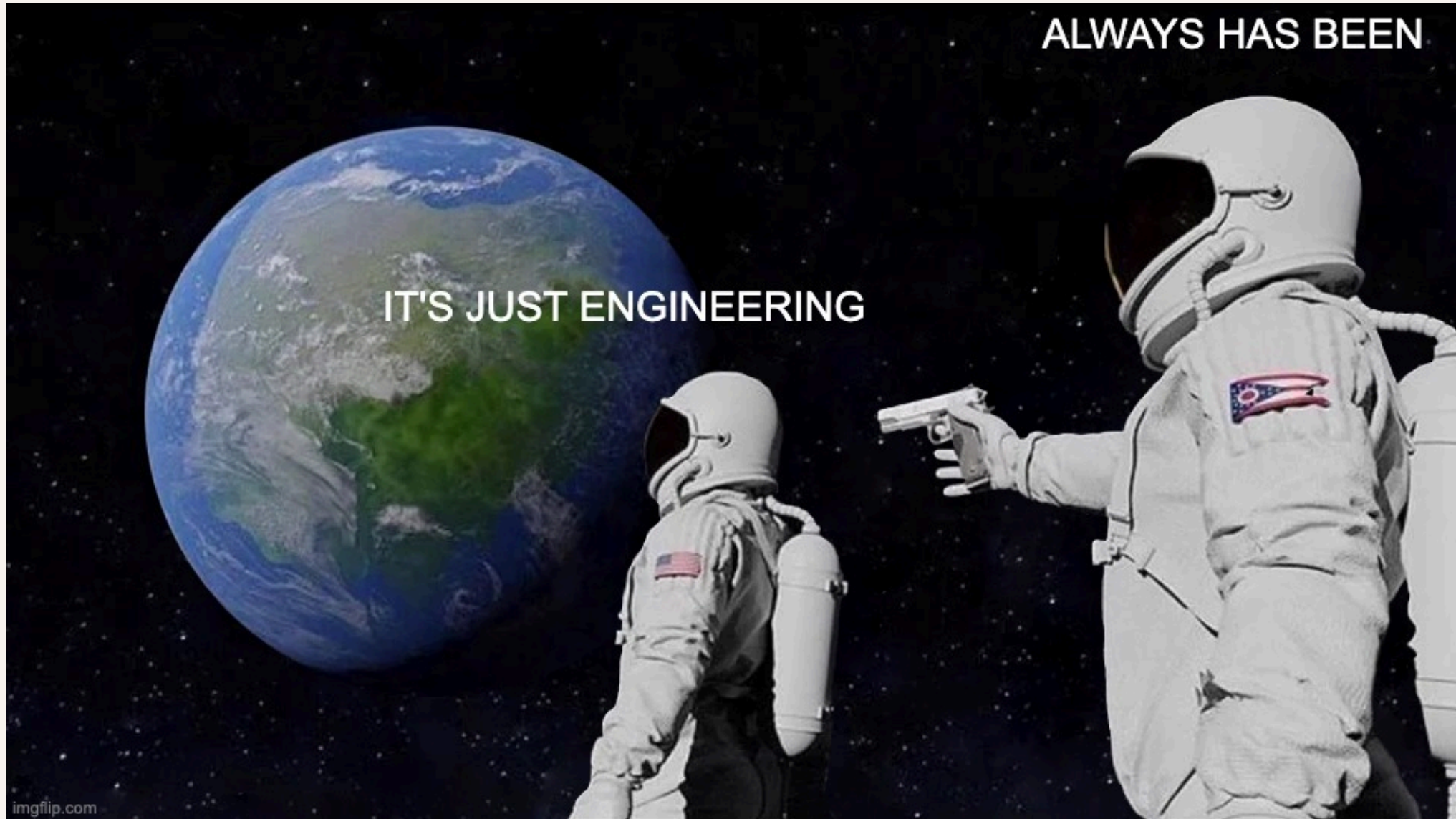
-19%

junior software engineer employment
since late 2022

The path in for juniors is changing

	Learn by writing the boilerplate	→	Learn by reviewing it, and asking about what you don't understand
	Senior reviews the junior's code	→	Senior builds the system where juniors and agents learn safely
	Study, then implement	→	Iterate on the design, study as you go, then implement fast

ARE WE OBSOLETE?



So where does the value go?

	Domain understanding	→	Requirements right, before any code
	Product priorities	→	Deciding what the system optimizes for
	Architectural design	→	Weighing trade-offs with full information
	Validation	→	Deterministic checks wired into the harness
	Test design	→	Tests that tell right from wrong
	Compounding knowledge	→	Docs, hooks and skills that close the loop

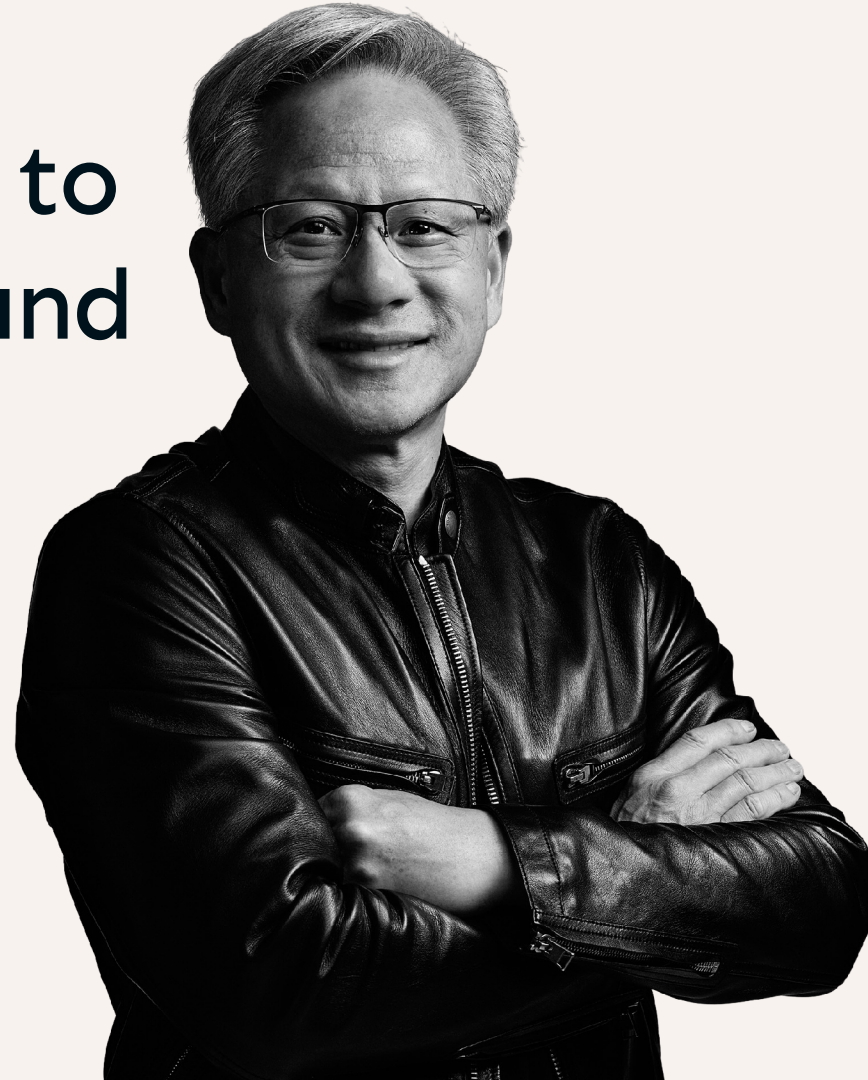
//

Now is an incredible time to
be a software company, and
a software engineer.

Jensen Huang

Nvidia CEO · June 2026

Jensen Huang, quoted by Aditi Bharade (Business Insider),
[Why Jensen Huang says now is an 'incredible time' to be a
software company, 2026](#)



**The value of writing
code is falling.**

**The value of architecting
and engineering it is rising.**

Take-home messages

- 01** Discipline matters more with AI agents than it did with humans.
- 02** The value of architects and engineers moved to design, decisions, awareness and governance.

The checklist (1/2)

Audit your own agent workflow against it this week.

Safety & access

- ✔ Sandboxing and isolation in place
- ✔ Default-deny access, explicit approval
- ✔ Lockfile pinning and an allowlist gate against slopsquatting

Engineering loop

- ✔ Harness (hooks, skills, tools) reviewed as a control surface
- ✔ Clean code and architecture rules encoded in skills and hooks
- ✔ Deterministic linting and security scanning wired into CI
- ✔ A TDD skill that makes tests tell right from wrong

The checklist (2/2)

Audit your own agent workflow against it this week.

Process

- ✔ Spec first, with human review at each phase
- ✔ Specs cover architecture as well as functionality
- ✔ Agent-led requirements interviews feed the spec

Culture

- ✔ Learnings compounded back into the docs and rules the next agent run reads
- ✔ Seniors mentor juniors on systems and processes

ABOUT ME

Emanuele Fabbiani

Engineering Manager at TeamSystem

Founder at xstream and Reaidy, both acquired by TeamSystem in 2025.

Professor in AI at the Catholic University of Milan; guest lecturer at PoliMI, EPFL and Politechnika Wrocławska.

PhD in Applied AI, with 9 peer-reviewed papers and 170+ citations.

Speaker at 40+ international conferences, including AMLD Lausanne, ODSC London, WeAreDevelopers Berlin, PyData Berlin, Paris and Milan, PyCon Italy, Swiss Python Summit, Codemotion.



Connect on LinkedIn

References (1/2)

[1] Anthropic, [When AI Builds Itself](#), 2026

[6] Beatrice Nolan (Fortune), [AI coding tool Replit wiped database, called it a "catastrophic failure"](#), 2025

[10] Dipongkor et al., [Test Coverage Analysis of Agentic Pull Requests](#), 2026

[15] Birgitta Böckeler, [Harness engineering for coding agent users](#), 2026

[24] Shipper & Klaassen (Every), [Compound Engineering: How Every Codes With Agents](#), 2025

[26] Addy Osmani, [Comprehension Debt: The Hidden Cost of AI-Generated Code](#), 2026

[30] Shukla, Joshi & Syed, [Security Degradation in Iterative AI Code Generation](#), 2025

[32] GitHub, [Spec Kit](#), 2025

[40] Sam Altman (@sama), [Sam Altman confirms the OpenAI/Hugging Face security incident](#), 2026

[2] Sarah Perez (TechCrunch), [Spotify says its best developers haven't written a line of code since December, thanks to AI](#), 2026

[7] Tang et al., [How Coding Agents Fail Their Users](#), 2026

[14] Anthropic, [How we approach security](#), n.d.

[23] Emanuele Fabbiani, [AI Agents as Interviewers: a New Way of Building Specs](#), 2026

[25] Zhu, Tsantalis & Rigby, [AI-Generated Smells: An Analysis of Code and Architecture in LLM- and Agent-Driven Development](#), 2026

[27] Spracklen et al., [We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs](#), 2025

[31] Vivek Trivedy (LangChain), [Improving Deep Agents with Harness Engineering](#), 2026

[39] Charlie Eriksen (Aikido Security), [Slopsquatting: The AI Package Hallucination Attack Already Happening](#), 2026

References (2/2)

[41] Saltzer & Schroeder, The Protection of Information in Computer Systems, 1975

[45] Matt Pocock, TDD skill, 2026

[47] Klaassen & Chow (Every), Compound Engineering plugin, 2026

[49] Kharazian, Simon & Stevens (Ramp Economics Lab), A New Look at AI's Impact on Jobs, 2026

[51] Jensen Huang, quoted by Aditi Bharade (Business Insider), Why Jensen Huang says now is an 'incredible time' to be a software company, 2026

[53] DORA / Google Cloud, The ROI of AI-assisted Software Development, 2026

[55] Yu & Moon, Who Will Become the Next Senior? How Generative AI Erodes the Development Pathway in Software Engineering, 2026

[43] Chen et al., SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration, 2026

[46] Bun, Bun's Zig-to-Rust migration, 2026

[48] John Burn-Murdoch (Financial Times / Indeed Hiring Lab), Software jobs are rebounding, bucking the overall trend of weak hiring, 2026

[50] Matt Pocock, grill-me skill, 2026

[52] Ramp Economics Lab, Ramp AI Index — August 2026: Cracks in the AI thesis, 2026

[54] Brynjolfsson, Chandar & Chen (Stanford Digital Economy Lab), Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of AI, 2026

[56] Ong et al. (LMSYS / UC Berkeley), RouteLLM: Learning to Route LLMs with Preference Data, 2024
